

qc-hardening

A 14-pass production audit that finds and fixes defects without changing functional behavior.

Use after feature work is complete, before production, release, merge, or security review.

Original: `~/claude/skills/qc-hardening/` · Suite: qc-all (shared ledger schema)

August 2026 · Port this document; do not weaken the laws.

Contents

- What it is, when to use it, and the seven laws
- Files to vendor
- Pre-flight
- The 14 passes
- Severity, evidence, ledger, and verdict math
- Hardening profile and pass debt
- Scaling: inline vs chunked vs Discovery+Verify
- Commits, anti-patterns, and the summary banner
- Minimum viable port

1. Purpose

qc-hardening is a systematic, multi-pass code audit. Each pass is additive and idempotent. Mechanical passes (1-11) clean noise; Carmack (12) and Chaos (13) find the bugs tools miss; Maintainability (14) turns evidence into an A-F judgment and a release verdict. It is not a code review, not a refactor, and not a feature factory.

Triggers: harden, production-ready, hardening pass, code audit, pre-production review, or “run this against the codebase” after feature work. **Do not use** during feature work, when architecture is the task, or on a throwaway prototype.

Seven laws — do not weaken these on a port

- Harden what exists. No features, no architecture, no public-API changes, no “while I’m here.”
- P2 is the floor for code changes. P3 is note-only. Never re-grade a style nit up to P2.
- A finding needs a concrete failure scenario. “Could theoretically...” is not a finding. Clean passes are expected.
- Fix in place or defer. A split, extract, or new module is never the minimal fix. Defer with `fixed:false + deferred_because`.
- Do not change tests to match buggy code. Prove the assertion wrong from an independent source, or leave it red.
- Detect-and-report is hardening; rewriting values (trim, canonicalize, default a field) is a feature.
- Any P0 blocks release even if fixed. That is a human sign-off gate, not a bug.

Agents invent principled-sounding exceptions. Port the rationalization table (section 8) verbatim. Three clean passes is the expected output of clean code, not a reason to manufacture findings.

2. Files to vendor

Path	Role
SKILL.md	Trigger, pre-flight, output contract, anti-goals. Canonical.
README.md	Human overview only. SKILL.md wins on conflict.
references/passes-1-11.md	Mechanical + coverage + docs + config.
references/passes-12-14.md	Carmack, Chaos, Maintainability.
references/severity-calibration.md	P0-P3, evidence formats, pass order.
references/pass-debt.md	Ledger lifecycle and .hardening-profile.md.
references/scaling-and-dispatch.md	Inline vs chunked; workflow vs subagents.
workflows/discovery-verify.js	Optional. Chunked Discovery+Verify.
qc-finding.schema.json (from qc-all)	Shared ledger schema. Copy if vendoring standalone.
verify_ledger.py (from qc-all)	Recomputes verdict. Fix the ledger, never the script.

If this skill is vendored without the qc-all suite, copy the schema and `verify_ledger.py` alongside it. Do not invent a different findings filename, JSONL, or per-pass files.

3. Pre-flight

1. Read AGENTS.md / CLAUDE.md / package.json. Project conventions override skill defaults.
2. `git log --oneline --grep=harden`. Prior clean runs are a positive signal.
3. Load .hardening-profile.md if present (hot spots, pass debt, Carmack manifest, clean streaks).
4. Detect language, package manager, CI, containers, secrets, public API, concurrency.
5. Install audit tools isolated (npx, pipx). Do not add them as project dependencies.
6. Run the full test suite. If it fails, stop. If there is no suite, file P1 and do not grade READY until one exists.
7. Record the starting commit SHA. Every pass commits independently for isolated revert.

Quick-check mode

A pass that has been clean for 3+ consecutive runs *and* has no new code in its scope may drop to tool-only. Eligible: 4, 5, 6, 7, 10, 11. **Never** quick-check: 1, 2, 3, 8, 9, 12, 13, 14.

Toolchain matrix (passes 4 and 8)

Language	Type checker	Linters	Coverage
Python	<code>mypy . --strict</code>	<code>ruff check . --fix && ruff format --check .</code>	<code>coverage run -m <runner></code>
TypeScript	<code>npx tsc --noEmit</code>	<code>npx eslint .</code>	<code>project test runner --coverage</code>
Rust	<code>cargo check</code>	<code>cargo clippy -- -D warnings</code>	<code>cargo tarpaulin</code>
Go	<code>go vet ./...</code>	<code>staticcheck ./...</code>	<code>go test -cover ./...</code>

4. The 14 passes

Run in order. Mechanical first so Carmack and Chaos do not waste time on lint. Pass 8 runs twice: 8a before the deep passes, 8b after. Do not batch-skip execution; you may batch consecutive clean commits.

#	Name	What it does	Skip?
1	Correctness	Deep-read returns, conditionals, loops, strings, numerics, collections, coercion. Then sequence-level (write-then-read, RMW atomicity, cleanup on all paths). Then API-parity across dual paths, field-population of output types, classification completeness, I/O type coercion at every ingest path. Regression test per fix.	Never
2	Security	Secrets grep + git history + Docker. Injection data-flow (SQL, shell, path, template, SSRF, pickle/yaml/eval, log injection). Ecosystem audit. Previously committed secrets = P0.	Never
3	Error handling	Inventory every external boundary. Bare except / no timeout = P1. Resource lifecycle. Schema drift. UnicodeDecodeError is not OSError.	Never
4	Types / lint	Typechecker + linter. Do not mass-reformat a repo that never formatted. Only format files this run touched.	Never (tool)
5	Concurrency	Shared state, lock ordering, missing await, check-then-act, RMW.	No concurrency signals
6	Performance	Algorithmic / I/O / unbounded memory with evidence. "Might be slow" is not a finding.	Never
7	Logging	Caught exceptions logged; no secrets/PII; no print() in library code. CLI print() is exempt.	Never
8	Coverage	Writes tests only. 8a: core <80%, error paths, edges. 8b after 12-13: every Carmack/Chaos fix has a regression test.	Never
9	Docs	Docs match code, not the reverse. Dead comments, TODOs, commented-out code out.	Never
10	Backward compat	Diff vs starting SHA. Must not change public signatures, error types, exit codes, serialized names. New validation stays as permissive as production currently is.	No public API
11	Env / config / build	Every env read documented; missing-key crash = P1; unsafe default (DEBUG=true) = P0. Lockfile, unused deps, CI/local parity, container (no :latest, non-root).	Never
12	Carmack	No checklist. Coverage manifest first (untested lines first). Five questions per function. Multi-order analysis of the ledger. Incomplete until every module is EXAMINED or FINDING.	Never
13	Chaos	Tests only. Adversarial inputs. Assert no-crash / right exception / graceful degradation — not specific output. Every crash: Carmack escalation (grep the pattern, fix all instances in one commit).	Never
14	Maintainability	After 13, not before. Six criteria, A-F grades. Findings need a concrete change-risk. Local, behavior-preserving fixes only; structural debt is deferred.	Never

Pass 1 extras that ports usually drop — do not drop them

- **1.2b Sequence-level:** Pass 1 historically misses bugs Carmack catches because it asks "is this function correct?" instead of "do these functions compose?"
- **1.2c API parity:** dual public paths (e.g. normalize-only vs full pipeline) must accept the same metadata and populate the same output fields.
- **1.2d Field population:** every output field with a None/empty default is either set on every constructor path, documented optional, or a latent bug. Helpers that exist but are never called count.
- **1.2e Classification completeness:** the bug is often in what a set contains, not how it is used.
- **1.2f I/O type coercion:** cleaning utilities must run on every ingest path, not just the primary one.
- **Cross-backend:** after every Pass 1 or 12 fix, grep other backends for the same pattern and fix them in the same commit.

Pass 12 — Carmack Review

This pass does not follow a checklist. It reads the codebase as a connected system. Step 0: run coverage and build an audit manifest (module, LOC, coverage, untested lines, status NOT YET / EXAMINED / FINDING). Untested lines first — they have never been proven. Step 0b: read the findings ledger and do first/second/third-order analysis (same pattern elsewhere? did the fix add an assumption? systemic gap?).

Five questions, every function, data flow, and state transition:

1. What invariant makes this correct? If you cannot state it, the code may be correct by accident.
2. What happens when my assumptions are wrong? Name them, then break each one.
3. What sequence of valid operations produces an invalid state?
4. If this runs a million times, what breaks? (Accumulated state, leaks, wraparound — not Pass 6 performance.)
5. What would make this code wrong tomorrow? Implicit coupling, magic strings, coincidences in current data.

Look specifically for: state corruption across calls, silent data loss, unenforced ordering, implicit contracts, degenerate inputs that pass validation. Do not report style, hypothetical future requirements, micro-optimization, or anything Passes 1-11 already caught. Typical: three Carmack runs; run 3 clean across all modules means well-hardened. Completion: every module EXAMINED or FINDING; every untested range explicitly reasoned about; CLEAN list names every function examined without findings. If you cannot state what you examined, you did not examine it.

Pass 13 — Chaos Monkey

Carmack reasons. Chaos *runs*. This pass writes tests only. Findings are reported like any other pass; the production fix lands in a subsequent commit. Dedicated permanent file (tests/test_chaos.py or project equivalent). Naming: test_chaos_<module>_<category>_<case>.

Category	Throw	Looking for
Encoding	Non-ASCII, null bytes, RTL, zero-width, emoji, surrogates	Crashes, mojibake, silent truncation
Boundaries	Empty, 10K+ chars, max-int, negative, NaN, Inf	Index/overflow, infinite loops, memory
Type confusion	None/int where string, list where dict, nested nulls	TypeError, silent wrong-type propagation
Injection-as-crash	SQL fragments, shell metacharacters, percent-s, brace-zero, ../	Crashes and corruption (not Pass 2 exploit)
Structural	Deep nesting, cycles, duplicate keys, extremely wide rows	RecursionError, KeyError, exhaustion
Timing / ordering	Double-call, wrong order, empty then large batch	State corruption, stale caches
Real-world garbage	BOM, mixed delimiters, N/A in numeric columns, dates as numbers	Production-shaped bugs

Assert no crash, the right exception, or graceful degradation. Do **not** assert specific output values for garbage input. When a chaos test crashes: file the finding, keep the test, **Carmack-escalate** (grep the pattern — a single instance is a bug, two are a systemic gap), fix all instances in one commit. Optional property-based testing (Hypothesis / fast-check / proptest) if already present; do not add it as a dependency. Also write pipeline-chain, stateful/idempotent, timeout-bound, and roundtrip tests.

Pass 14 — Maintainability

Do not run this early; judgments before the deep passes are taste. Score six criteria: standards discipline; module boundaries; local reasoning cost; typed/validated contracts; testability; operational/doc consistency. Grades: A (local changes stay local) through D/F (routine mods need whole-system context). P1 = concentrated change-risk; P2 = local inconsistency with a safe fix; P3 = style / large refactor — do not fix. Allowed to end with deferred findings. Required output: Consistency, Maintainability, Overall grades; named hot spots; release-ready / releasable-with-debt / not-yet.

5. Severity, evidence, ledger, verdict

Sev	Meaning	Action
P0 — Outage	Data loss, breach, or production service failure	Fix now. Blocks release even if fixed.
P1 — Defect	Incorrect behavior under realistic conditions	Fix this pass, or defer with reason.
P2 — Weakness	Edge / concrete risk, cheap to fix	Fix if low-risk; else note.
P3 — Style	Style, minor inconsistency, theoretical concern	Do not fix. Note only if systemic.

Confidence when prioritizing: Carmack/Chaos P1 (proven) > mechanical P1 (pattern-matched). The bar for filing: you must describe a concrete scenario. **Full evidence** (passes 1, 2, 3, 12, 13): id, file:line, what, why, severity, fix. **Lightweight** (tool passes 4-11): id, file:line, what (tool output), fix — the tool is the why. IDs: F<pass>.<n> (F12.3).

Ledger contract

Write one JSON object to `.qc-findings/qc-hardening.json`. Do not split per pass. Do not use JSONL. After every pass, rewrite the full accumulated object via temp file + `rename()` so a concurrent reader never sees a partial write. Required top-level: `schema_version` "1.0", `skill` "qc-hardening", `run_id` (YYYY-MM-DDTHH:MM:SSZ-<7-char-sha>), `git_sha`, `findings[]`, `verdict`. Required per finding: id, severity, file, what, fixed. Carry pass, line, why, fix, scope where known.

A deferral is **only** fixed: `false` **and** a non-empty `deferred_because`. Any other encoding of “later” counts as an open P1 and blocks release.

```
{
  "schema_version": "1.0",
  "skill": "qc-hardening",
  "run_id": "2026-05-17T14:30:00Z-abc1234",
  "git_sha": "abc1234",
  "findings": [{ "id": "F12.3", "pass": 12, "scope": "module",
    "severity": "P1", "file": "app/foo.py", "line": 42,
    "what": "Race in concurrent write path",
    "why": "Two threads pass _closed before either acquires the lock",
    "fix": "Move _closed check inside the lock", "fixed": true }],
  "verdict": "READY_WITH_DEBT"
}
```

Verdict algorithm (must match `verify_ledger.py`)

```
if any P0 exists (fixed or not)      -> NOT_READY
else if any open P1                  -> NOT_READY  # fixed=false, no deferred_because
else if any P1 exists (all fixed or deferred) -> READY_WITH_DEBT
else if any open P2                  -> READY_WITH_DEBT
else                                  -> READY
```

SKIPPED is reserved for qc-all rollup. A hardening run that executes never emits it.

Before the summary, run `verify_ledger.py` against the ledger. Mismatch: fix the ledger, not the script. A run that hit and repaired an outage-class bug still grades NOT_READY so a human clears the release.

6. Hardening profile and pass debt

Commit `.hardening-profile.md` in the repo. Update it incrementally so a crashed run is not lost. Sections: run history; maintainability trend; pass streak (feeds quick-check); hot spots (read first next time); **pass debt** (“Pass 12 found X that Pass 1 should have caught” + why); Carmack audit manifest (EXAMINED / FINDING / NOT YET). Later runs only deep-read NEW/CHANGED + remaining NOT YET. Same module in pass debt 3+ times: dispatch a dedicated reader. On re-run, previously committed fixes are not re-applied. The summary reports this run only.

7. Scaling

- **Inline** (default, ~<5K LOC): coordinator reads modules itself.

- **Chunked** (>5K LOC, >~8 modules, or you are skimming): partition into 500-800 LOC groups; keep coupled modules together.
- **Partial re-run**: if >80% of modules are already EXAMINED and new+changed LOC ~<2K, deep-read only the delta + hot spots.

In chunked mode, Discovery+Verify (the read half of passes 1, 2.2, 3, 5, 12) is a workflow (workflows/discovery-verify.js): Deep-Read one agent per group (schema-enforced findings + CLEAN list — empty findings + full CLEAN is valid); Verify with two adversarial votes per finding; Synthesize (dedup, promote confirmed cross-module concerns, rank P0>P1>P2). Returns recommendation fix / defer / drop. **The workflow never edits or commits.** Remediation stays sequential in the parent: apply, regression-test, narrow gates + golden suite, isolated commit (never git add -a if in-flight files exist). Cross-module findings are first-class (F-ID, severity, evidence) — not unnamed “concerns.”

8. Commits, anti-patterns, summary

Commit rules

- Run passes in order. Each pass commits before the next begins.
- Format: harden: pass N — <name> (F2.1 F2.3) or (0 fixes) if clean.
- Batch consecutive clean passes: harden: passes 3,5,6,7 — clean. Zero-diff clean passes need no commit.
- If a fix creates a test regression, fix it in the same commit. Update broken tests to the new correct behavior — do not preserve the old expectation.

Rationalization table (port verbatim)

Excuse	Reality
The tangled structure is the defect, so restructuring is the minimal P1 fix.	A split is never the minimal fix. Patch in place or defer.
The reviewer always blocks this style nit, so it is really a P2.	Severity measures the running system. Naming/style is always P3.
Three clean passes looks lazy — I will file borderline things at P2.	Fabricating severity is fabricating a finding. Three zeros is expected.
Changing a default from [] to None is behavior-preserving.	A public default is part of the signature. Defer it. Private functions: fix in place.
Release is in N minutes; the test is probably stale.	A deadline is not evidence. The code cannot prove its own test wrong.
Pass 3/13 say add validation, so I will trim/canonicalize the input.	Detect-and-report is hardening. Rewriting values is a feature. File and defer.
The only real fix for this Carmack P1 is new architecture, and P1 means fix-this-pass.	A finding whose only fix is new architecture is a deferred finding. Record it; do not refactor.

Red flags — STOP and return to the current pass

- Rewriting a function “for clarity” that has no defect; adding a module, class, or abstraction.
- Changing public API signatures or adding configuration options; refactoring test infrastructure.
- Writing a test for a scenario that cannot happen given the current code.
- Rabbit-holing on a single P2 — one focused read, then note and move on.

Summary banner (exact)

After all passes, emit HARDENING SUMMARY with: start/final SHA, passes run (note SKIPPED or quick-checked), lines changed, per-pass finding counts, A-F scorecard, finding index (F1.1 (P1) — one-liner — fixed in <sha>), totals, clean-pass count, deferred list, remaining human concerns, suite status. Any P0 or open P1: the banner must say **RELEASE BLOCKED** and verdict must be NOT_READY.

9. Minimum viable port

Must have, or it is not this skill	Can stub on first port
14-pass order; P0-P3 + evidence bar	Discovery-verify workflow (serial reads instead)
Atomic ledger + verdict math + deferred_because	Hypothesis / property-based chaos
Per-pass isolated commits	Autonomous-supervisor dispatch for leftover P1s
Carmack 5Q + Chaos + Pass 14 synthesis	qc-all rollup integration
No-features / no-architecture laws + profile + pass-debt	

Do not invent: a different findings filename, JSONL, per-pass files, “open P0 only blocks,” or a READY grade while P0s sit in the ledger as fixed:true. On LTBot this skill ran nine times: mechanical passes were already green; almost all value came from Pass 12 on live-adapter paths the in-memory suite cannot see.